

NovaBet

Dossier Projet

SPRINT 2 — ENRICHISSEMENT

Hugo · Siloé · Antonio · Kahina | Mai 2026

1. Présentation du projet

Nom du projet : NovaBet

Concept : Application de bureau Java/JavaFX fusionnant les marchés de prédiction (*prediction markets*) et des mécaniques de casino. L'utilisateur prédit l'issue d'événements réels (sport, politique, crypto, IA...), gagne ou perd de la monnaie fictive, et se voit proposer de convertir ses gains en crédits casino via une offre unique au moment du retrait.

Proposition de valeur : Un funnel de monétisation agressif — l'OTO (*One Time Offer*) — qui convertit les gains de marchés en crédits casino au moment du retrait, générant ainsi un engagement prolongé et une rétention maximale.

Cible (ICP) : Jeunes adultes (18-35 ans), familiers de l'écosystème Crypto/DeFi, déjà utilisateurs de plateformes de paris sportifs, de trading ou de marchés de prédiction.

2. Organisation de l'équipe

Rôles Sprint 2

Membre	Rôle	Responsabilités
Hugo	Scrum Master	Organisation, cérémonies Scrum, coordination, suivi sprint
Siloe	Product Owner	Vision produit, priorisation backlog, validation features
Antonio (Jusce)	Développeur	Logique métier, fusion des packages, intégration UI
Kahina	Lead Développeuse	Architecture, modélisation BDD, migrations, DAOs

Les rôles tournent à chaque sprint.

Stack Technique

- Java 21 / JavaFX 21
- Maven
- MySQL 8 / JDBC
- BCrypt (jBCrypt 0.4) — hashage des mots de passe
- JUnit 5 — tests unitaires
- **Mockito 5.11** — mocks pour les tests de services à transactions atomiques (*nouveau Sprint 2*)
- **Stripe Java SDK 28.0.0** — Identity (KYC) et Checkout (dépôts) (*nouveau Sprint 2*)
- Git / GitHub (GitFlow : main, develop, feature/*)
- Gestion de projet : Trello

3. Backlog & Planification

Sprint 1 — Cadrage + MVP

ID	User Story	Statut
US-01	En tant qu'utilisateur, je peux créer un compte	✓ Fait
US-01b	En tant qu'utilisateur, je peux me connecter	✓ Fait
US-02	En tant qu'utilisateur, je peux déposer de la monnaie fictive	✓ Fait
US-03	En tant qu'utilisateur, je peux voir la liste des marchés ouverts	✓ Fait
US-04	En tant qu'utilisateur, je peux acheter des parts YES/NO	✓ Fait

Sprint 2 — Enrichissement (Logique Métier)

ID	User Story	Statut
US-05	En tant qu'utilisateur, je peux voir mon portefeuille et mes positions en cours	✓ Fait
US-06	En tant qu'utilisateur, je peux consulter l'historique de mes transactions	✓ Fait
US-07	En tant qu'utilisateur, je peux créer un nouveau marché de pari	✓ Fait
US-08	En tant qu'admin/système, je peux résoudre un marché (YES/NO) et payer les gagnants	📦 Logique métier livrée, finalisation Sprint 3

Bonus livré Sprint 2

Élément	Statut
Onboarding NovaBet multi-étapes + KYC Stripe Identity	✓ Fait
Dépôts via Stripe Checkout (mode test)	✓ Fait
Casino Lobby (FXML) — topbar, ribbon wagering, filtres SLOT/CRASH, grille dynamique	✓ Fait
One-Time Offer — backend complet + roue de la fortune (UI)	✓ Fait
Refonte UI unifiée "Black & Gold" sur toutes les vues	✓ Fait
Fusion des packages <code>com.novabet</code> → <code>com.polymarket</code>	✓ Fait

Sprint 3 — Finalisation & Funnel Casino

- Implémentation des jeux du casino (slot machine + crash game) jouables depuis le lobby
- Business plan général (stratégies d'acquisition, monétisation, positionnement marché)
- Finalisation du règlement des marchés (paiement automatique des gagnants)
- Extension de la couverture de tests

- Polishing UI final et préparation de la soutenance

4. Avancement Sprint 2

Avancement global estimé : 93%

Ce qui fonctionne (end-to-end)

- **Onboarding NovaBet complet** : inscription multi-étapes (email/mot de passe → CGU → vérification d'identité KYC → premier dépôt) entièrement branchée sur Stripe Identity et Stripe Checkout. Polling en arrière-plan, mise à jour transactionnelle de `users.kyc_status` et insertion dans `transactions`.
- **Portfolio utilisateur** : affichage des paris de l'utilisateur connecté, jointure `bets` → `outcomes` → `events`, navigation fluide Portfolio ↔ Markets / Wallet / History / Create Market.
- **Wallet** : UI complète avec solde réel, solde virtuel et total. Dépôts et retraits (monnaie fictive `$NVB`), historique des transactions, service métier `WalletService` + exceptions dédiées (`InvalidAmountException`).
- **History** : écran dédié listant paris et transactions.
- **UI Markets refaite** : `MarketDetailView` refondue, sidebar de navigation unifiée sur tous les écrans, refresh dynamique du solde en topbar.
- **Casino Lobby** : lobby FXML avec hero ribbon affichant la progression du wagering, bouton cashout, filtres SLOT / CRASH, grille de jeux chargée depuis la BDD. Branding NovaBet, interface en français.
- **WageringService** : règle de mise 1× appliquée (`canCashout`, `recordBet`, `recordWin`) en transaction SQL atomique avec rollback sur échec.
- **CasinoCashoutService** : transfert `virtual_balance` → `real_balance` atomique avec reset du compteur de wagering.
- **OTO (One-Time Offer)** : backend complet — modèle `OtoEvent` / `CasinoSession`, DAOs, `OtoService` avec acceptation transactionnelle (event + session, tout-ou-rien). Roue de la fortune (`WheelLogic` testable + `WheelController` JavaFX) câblée à l'UI.
- **Tests unitaires** : 66 tests passent (`BettingServiceImpl`, `WageringService`, `OtoService`, `CasinoCashoutService`, `WheelLogic`).
- **Migrations BDD** : 3 migrations appliquées (casino lobby + `wagered_amount`, renommage FR des jeux, KYC + colonnes Stripe).

Ce qui est en cours

- **Règlement des marchés (US-08)** : logique métier livrée côté `BettingService`, finalisation du paiement automatique des gagnants prévue Sprint 3.
- **Couverture de tests** : 66 tests sur les services critiques — extension prévue Sprint 3 sur les DAOs et controllers.
- **Implémentation des jeux du casino** : lobby et catalogue livrés, mais les jeux eux-mêmes (slot, crash) ne sont pas encore jouables — prévu Sprint 3.

Chiffres clés du sprint

- ~14 commits sur `develop` (8 features + 6 merges de branches)

- ~96 fichiers modifiés/ajoutés sur la période
- ~6 535 lignes ajoutées / ~567 lignes supprimées
- 66 tests unitaires (JUnit 5 + Mockito)
- 3 migrations BDD
- 2 nouvelles dépendances Maven (Mockito, Stripe SDK)

5. Difficultés rencontrées

Niveau	Problème	Impact
● MAJEUR	Cohabitation des packages <code>com.polymarket</code> et <code>com.novabet</code> pendant la phase d'intégration	Deux entrypoints distincts, double maintenance — résolu en fin de sprint par la fusion
● MAJEUR	Hétérogénéité des palettes UI entre les vues (héritée du Sprint 1)	Expérience utilisateur incohérente — résolue par la refonte design unifiée
● MOYEN	Coordination du merge final entre les branches feature	Plusieurs conflits Git à résoudre, retard d'environ une journée sur la fin de sprint
● MOYEN	Suppression du <code>module-info</code> côté NovaBet pour permettre l'exécution classpath (Stripe SDK non modulaire)	Refactor de l'entrée de l'application (<code>Launcher</code>), arbitrage technique en cours de sprint
● MOYEN	Configuration Surefire à adapter pour Mockito sous JDK récent (<code>-XX:+EnableDynamicAgentLoading</code>)	Heure perdue sur la configuration de la chaîne de tests

6. Décisions prises

Date	Décision	Raison
Avril 2026	Architecture MVC + Hexagonale	Isolation de la logique métier pour permettre les tests sans BDD
Avril 2026	BCrypt pour le hashage	Sécurité : jamais de mot de passe en clair en base
Avril 2026	Validation regex côté front	UX : erreurs immédiates sans round-trip serveur
Mai 2026	Renommage com.polymarket → com.novabet (initial)	Cohérence du projet suite au rebranding
Mai 2026	Adoption de Mockito pour les tests de services à transactions atomiques	Permettre de tester WageringService, OtoService, CasinoCashoutService sans BDD réelle, en mockant <code>Connection</code> et les DAOs
Mai 2026	Stripe Identity pour le KYC + Stripe Checkout pour les dépôts	Solution éprouvée, conforme PCI, évite la manipulation directe de documents d'identité et de cartes
Mai 2026	Unification design "Black & Gold" sur toutes les vues	Résolution de l'hétérogénéité UI héritée du Sprint 1, identité visuelle unifiée
Mai 2026	Pattern Wallet → OTO → Casino : le retrait passe désormais par une roue de la fortune avant validation	Cœur du modèle de monétisation NovaBet — l'OTO transforme l'intention de retrait en engagement casino
Mai 2026	Suppression définitive du package com.novabet, tout dans com.polymarket	Un seul entripoint, un seul module — fin de la cohabitation héritée du Sprint 1

7. Rétrospective Sprint 2

Ce qui a bien marché

- Intégration continue cette fois-ci : merges réguliers sur `develop` tout au long du sprint
- Refonte design unifiée réussie — l'application a désormais une identité visuelle cohérente sur toutes les vues
- OTO bien câblée — backend, UI et logique de la roue de la fortune intégrés en transaction atomique
- 66 tests unitaires sur les services critiques (WageringService, OtoService, CasinoCashoutService, WheelLogic, BettingService)
- Adoption de Mockito qui débloquent les tests de services à transactions atomiques
- Stripe Identity + Stripe Checkout intégrés proprement avec configuration externalisée (`config.properties` ignoré par git, `.example` versionné)

Ce qui n'a pas marché

- Retard sur la fusion des packages — la cohabitation `com.polymarket` / `com.novabet` a duré jusqu'en fin de sprint
- Quelques filtres UI cosmétiques (filtres SLOT/CRASH du lobby) sont visuellement présents mais non fonctionnels — à terminer Sprint 3
- Les jeux du casino eux-mêmes ne sont pas encore jouables — le lobby est livré mais les jeux restent à coder
- Couverture de tests encore concentrée sur les services — les DAOs et controllers n'ont pas de tests à ce stade

Actions Sprint 3

- Implémenter les deux jeux jouables (slot machine + crash game) depuis le lobby
- Rédiger le business plan général (acquisition, monétisation, positionnement marché)
- Étendre la couverture de tests aux DAOs et controllers
- Finaliser le règlement automatique des marchés
- Polishing final + préparation de la soutenance

8. Budget Sprint 2

Budget initial : 100 000 € Budget restant après Sprint 1 : 88 900 €

Rôle	Tarif	Jours	Coût Total
Product Owner	1 000 €/j	1 jour	1 000 €
Scrum Master	800 €/j	1 jour	800 €
Dev Front	400 €/j	6 jours	2 400 €
Dev Full Stack	550 €/j	14 jours	7 700 €
Dev Back	600 €/j	8 jours	4 800 €
Testeur	800 €/j	1 jour	800 €
TOTAL		31 jours	17 500 €

Budget consommé Sprint 2 : 17 500 € Budget restant après Sprint 2 : 71 400 € (disponible pour le Sprint 3)

9. Projection Sprint 3

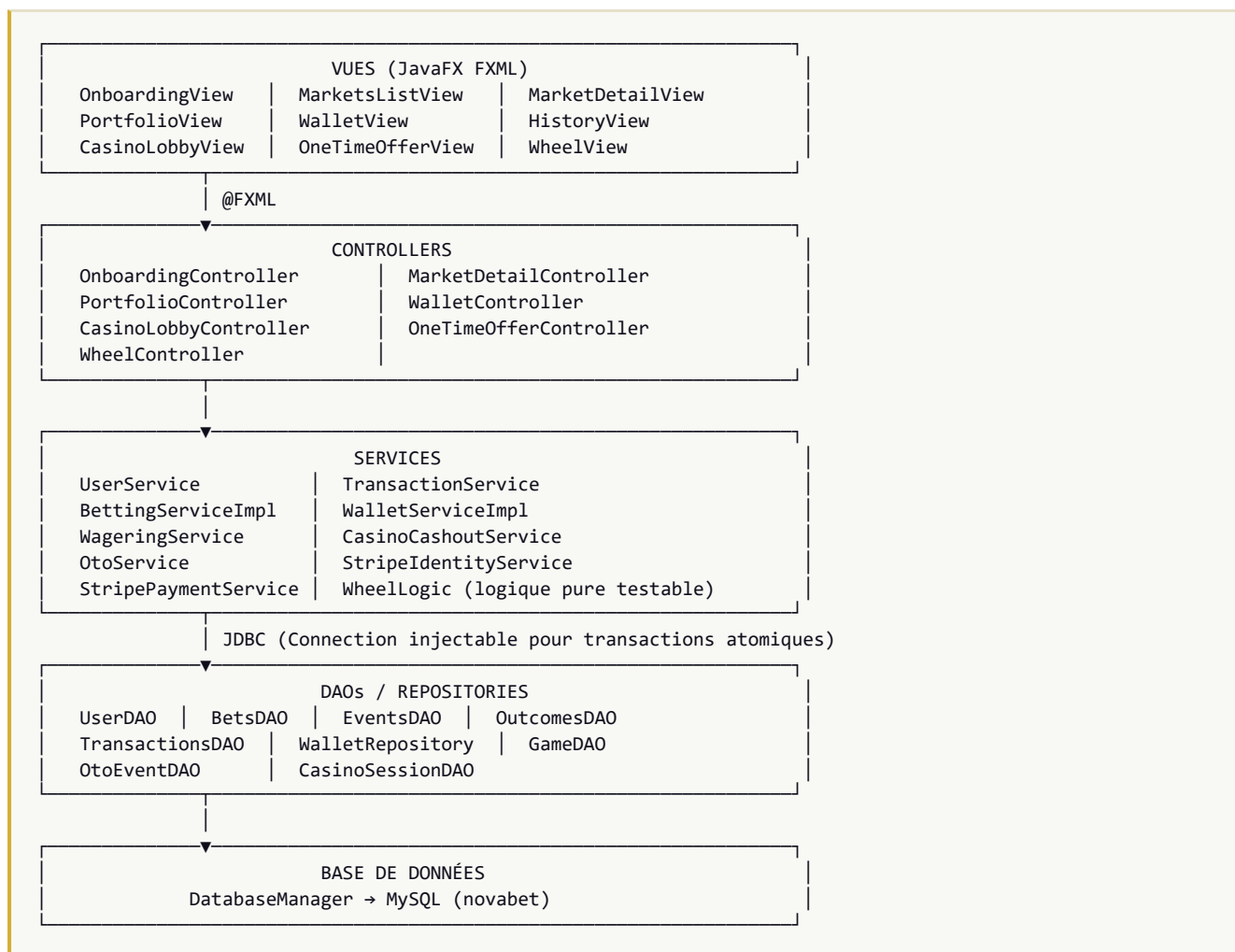
Budget prévisionnel Sprint 3 : à arbitrer une fois le Sprint 2 chiffré.

Priorités

Priorité	Tâche
P0	Création des deux jeux (slot machine + crash game) jouables depuis le lobby
P0	Business plan général : stratégies d'acquisition, monétisation, positionnement marché
P1	Couverture de tests étendue (DAOs, controllers, scénarios end-to-end)
P1	Finalisation du règlement des marchés (paiement automatique des gagnants)
P2	Préparation de la soutenance finale (slides, démo, dossier projet final)
P2	Polishing UI final + filtres cosmétiques restants du lobby casino

10. UML Global

Architecture applicative (enrichie Sprint 2)



Patterns clés ajoutés au Sprint 2

- **Transaction atomique JDBC** : services qui orchestrent `setAutoCommit(false)` → opérations → `commit` / `rollback` dans `finally` (`WageringService`, `CasinoCashoutService`, `OtoService`)
- **Double constructeur DAO/Service** : un constructeur production (ouvre sa propre connexion), un constructeur d'injection (`Connection` passée) pour les transactions multi-DAO et les tests Mockito
- **Séparation logique pure / contrôleur** : `WheelLogic` extrait de `WheelController` pour rendre la logique de tirage pondéré testable sans dépendance JavaFX
- **DTO immuables via Java record** : `WalletSnapshot`, etc.

11. MCD / MLD

MCD (entités et associations)

```

USER (id, username, email, password_hash, created_at,
      kyc_status, stripe_verification_session_id)
WALLET (id, real_balance, virtual_balance, wagered_amount)
TRANSACTION (id, type, amount, created_at,
              status, stripe_session_id, payment_method)
EVENT (id, title, description, status, resolution, created_at)
OUTCOME (id, label, odds)
BET (id, amount, potential_win, status, created_at)
GAME (id, name, type)
GAME_SESSION (id, bet_amount, result, win_amount, created_at)
CASINO_SESSION (id, user_id, status, started_at, ended_at)
OTO_EVENT (id, user_id, type, accepted_at)

POSSEDE      : USER (1,N) ↔ (1,1) WALLET
EFFECTUE     : USER (1,N) ↔ (0,N) TRANSACTION
PARIE        : USER (1,N) ↔ (0,N) BET
CONCERNE     : BET (1,N) ↔ (1,1) OUTCOME
APPARTIENT   : OUTCOME (1,N) ↔ (1,1) EVENT
JOUE         : USER (1,N) ↔ (0,N) GAME_SESSION
EST_SUR      : GAME_SESSION (1,N) ↔ (1,1) GAME
DECLENCHE    : USER (1,N) ↔ (0,N) OTO_EVENT
OUVRE        : USER (1,N) ↔ (0,N) CASINO_SESSION

```

MLD (tables relationnelles)

USER	(id PK, username, email UNIQUE, password_hash, created_at, kyc_status ENUM('pending','processing','verified','rejected'), stripe_verification_session_id)
WALLET	(id PK, real_balance, virtual_balance, wagered_amount, #user_id FK→USER)
TRANSACTION	(id PK, type, amount, created_at, status ENUM('pending','succeeded','failed'), stripe_session_id, payment_method, #user_id FK→USER)
EVENT	(id PK, title, description, status, resolution, created_at, #outcome_id FK→OUTCOME)
OUTCOME	(id PK, label, odds, #bet_id FK→BET)
BET	(id PK, amount, potential_win, status, created_at)
GAME	(id PK, name, type ENUM('SLOT','CRASH'))
GAME_SESSION	(id PK, bet_amount, result, win_amount, created_at)
CASINO_SESSION	(id PK, status, started_at, ended_at, #user_id FK→USER)
OTO_EVENT	(id PK, type ENUM('DISPLAYED','ACCEPTED','REFUSED','EXPIRED'), accepted_at, #user_id FK→USER)
EFFECTUE	(#user_id FK→USER, #transaction_id FK→TRANSACTION)
PARIE	(#user_id FK→USER, #bet_id FK→BET)
JOUE	(#user_id FK→USER, #game_session_id FK→GAME_SESSION)

Migrations appliquées Sprint 2

- V1__casino_lobby.sql — ALTER TABLE wallets ADD COLUMN wagered_amount DECIMAL(10,2) + seed des 2 jeux (Slots Diamant , Crash Roquette)
- V2__rename_games_fr.sql — francisation du catalogue
- 001_kyc_and_stripe.sql — extension users (kyc_status, stripe_verification_session_id) et transactions (status, stripe_session_id, payment_method)

12. Choix techniques

Choix	Justification
JavaFX + FXML	Imposé par le projet. FXML permet la séparation vue/logique et le data-binding via @FXML
Architecture MVC	Séparation claire des responsabilités (vue, contrôleur, service)
Architecture Hexagonale (BettingService)	Isolation de la logique métier — permet les tests unitaires sans base de données
BCrypt (jBCrypt)	Standard de sécurité pour le hashage des mots de passe. Salage automatique
MySQL + JDBC	Imposé par le projet. Config externalisée dans <code>config.properties</code>
PreparedStatement	Protection contre les injections SQL
Mockito 5.11 (S2)	Permet de tester les services à transactions atomiques (<code>WageringService</code> , <code>OtoService</code> , <code>CasinoCashoutService</code>) sans BDD réelle, en mockant <code>Connection</code> et les DAOs
Stripe Identity (S2)	KYC certifié, conforme PCI — alternative à un upload manuel de documents d'identité, polling de statut en arrière-plan
Stripe Checkout (S2)	Dépôts sécurisés via session hébergée (mode test pour la démo, crédite <code>virtual_balance</code>). Évite la manipulation directe des cartes bancaires
Pattern Transaction atomique JDBC (S2)	<code>setAutoCommit(false)</code> + <code>commit</code> / <code>rollback</code> dans <code>finally</code> — garantit l'intégrité sur les opérations multi-tables (acceptation OTO, cashout casino, recordBet wagering)
Pattern Double constructeur DAO/Service (S2)	Constructeur par défaut (production) + constructeur avec <code>Connection</code> injectée (transactions multi-DAO et tests Mockito)
Java record pour DTO immuables (S2)	<code>WalletSnapshot</code> et autres DTO — immutabilité, code concis, thread-safety par construction
JavaFX FXML + CSS unifié (S2)	Toutes les vues désormais en FXML + CSS centralisé "Black & Gold" — résolution de l'hétérogénéité du Sprint 1 (Java pur vs FXML)
Mode classpath (Launcher + suppression module-info côté NovaBet) (S2)	Le SDK Stripe n'est pas modulaire — passage en classpath via une classe <code>Launcher</code> séparée
<code>config.properties</code> externalisé + <code>.example</code> versionné (S2)	Clés API Stripe et identifiants BDD jamais commités, exemple fourni pour onboarding développeur

13. État de la documentation

Document	État
docs/dossier_projet.md	✓ Version Sprint 1 archivée
docs/dossier_projet_sprint2.md	✓ À jour — Sprint 2 (ce document)
README.md	✓ Instructions installation et lancement
bdd_novamarket.sql	✓ Schéma complet (étendu par les 3 migrations Sprint 2)
config.properties.example	✓ Versionné, clés Stripe / BDD documentées
Javadoc	✗ Non généré — prévu Sprint 3
Guide utilisateur	✗ Non rédigé — prévu Sprint 3
Guide d'installation	✓ Dans README.md

14. Déclaration d'usage de l'IA

L'intelligence artificielle a été utilisée dans ce projet de manière transparente pour les activités suivantes :

Usage	Outil	Détail
Brainstorming	Claude	Idéation du concept NovaBet, modèle OTO, proposition de valeur
Aide au code	Claude	Génération initiale de <code>UserService.java</code> , <code>TransactionService.java</code> , modifications de <code>OnboardingController.java</code>
Aide au code (S2)	Claude	Assistance sur l'intégration Stripe Identity / Checkout, squelettes de services à transactions atomiques, structuration des DAOs <code>OtoEventDAO</code> / <code>CasinoSessionDAO</code>
Refonte design (S2)	Claude	Génération de FXML/CSS pour la refonte UI unifiée "Black & Gold" et le casino lobby
Génération de tests (S2)	Claude	Squelettes de tests Mockito pour <code>WageringServiceTest</code> , <code>OtoServiceTest</code> , <code>CasinoCashoutServiceTest</code>
Aide à la documentation	Claude	Rédaction du dossier Sprint 1 puis du présent dossier Sprint 2
Support de soutenance	Claude	Génération du PowerPoint Sprint 1 Review puis du diaporama Sprint 2 Review

Tout le code généré par IA a été relu, compris et validé par les membres de l'équipe avant intégration.